

Overview

Elasticsearch as the primary search engine: custom analyzer, bool DSL, function_score boosting, completion suggester for autocomplete, and two sync strategies (Debezium CDC or TransactionalEventListener).

Inverted Index vs Table Scan

	Inverted Index (Elasticsearch)	Table Scan (PostgreSQL LIKE)
Complexity	O(1) term lookup	O(n) full scan
Full-text relevance	TF-IDF / BM25 scoring	None (only exact/prefix)
Fuzzy matching	Built-in (fuzziness:AUTO)	Only via pg_trgm extension
Facets / Aggregations	Native aggs	GROUP BY (limited)
Sync	Async (event/CDC)	Real-time (same DB)

Custom Analyzer: shopverse_analyzer

Character Filters – none (standard)

Tokenizer – standard (Unicode text segmentation)

Token Filters (in order):

lowercase – normalise case

stop – remove common English stop words

synonym – laptop ↔ notebook, phone ↔ mobile

Products Index Field Mapping

Field	Type	Analyzer / Notes
name	text	shopverse_analyzer; boost ^3 in multi_match
description	text	shopverse_analyzer; boost ^1 in multi_match
sku	keyword	Exact match; no analysis
price	scaled_float	scaling_factor: 100
tags	keyword	Array; used in filter clause
nameSuggest	completion	Feeds autocomplete suggester
avgRating	float	Used in function_score boost (>4 → ×1.5)

Search Query Architecture (ProductSearchService)

bool query

must → multi_match {name^3, description^1} (full-text)

filter → price range + tags + category (no score impact)

should → boost in-stock items

function_score wrapper (searchWithBoost)

avgRating > 4 → weight 1.5

stock > 0 → weight 2.0

Data Sync Strategies

Strategy	Class	Trigger	Latency
Debezium CDC	DebeziumConfig + ProductChangeEventProcessor	Kafka input replication slot → Kafka output	Near real-time
Event Listener	SearchSyncListener	@TransactionalEventListener(AfterCommit)	After Record Commit
Bulk Indexer	BulkProductIndexer	Manual / initial load	Batch of 500